

Aula 3

Programação II

Prof. Elton Masaharu Sato

Conversa Inicial

Conversa Inicial

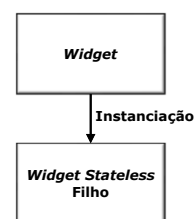
- *Stateful* e *Stateless*
- *Widgets* Básicos
- Montagem de interface com *Widgets*
- Entrada de Usuário com *Widgets*

Stateless e *Stateful*

Stateless

- Simples
- Tecnicamente Imutável (*build*)
- Ideal para *widgets* “fixos”

Stateless



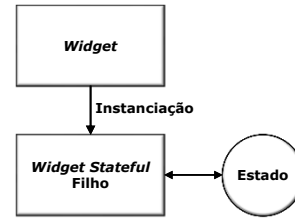
Fonte: Sato, 2022

Stateful

- Possui estado
- Mutável através do seu estado
- Ideal para *widgets* “dinâmicos”

7

Stateful



Fonte: Sato, 2022

8

Analogia



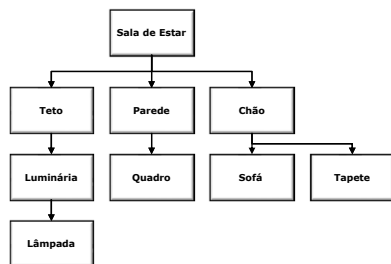
Photographie.eu/shutterstock

9

Árvore de *widgets*

- Um aplicativo é visto como uma árvore de *widgets*
- Um *widget* inicial cria outros *widgets*, que criam outros *widgets*...

10



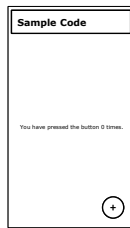
Fonte: Sato, 2022

11

Scaffold

12

Scaffold



Fonte: Flutter.dev

Scaffold

- "Andaime"
- *AppBar*
- Corpo central
- Barra de navegação inferior
- *Drawers*
- Botões, etc.

Código de um scaffold

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

class MyHomePage extends StatefulWidget {
  MyHomePage({Key key, required this.title}) : super(key: key);

  final String title;

  @override
  _MyHomePageState createState() => _MyHomePageState();
}

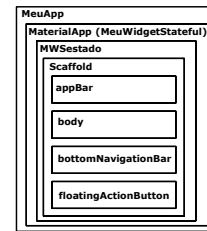
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      // This call to setState tells the Flutter framework that something has
      // changed in this state, which causes it to rerun the build method below
      // so that the display can reflect the updated values of the state. This is
      // the magic of Flutter: you put the state in the widget's class, and the
      // framework reruns the widgets below the state class whenever the state
      // changes.
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Text('You have pressed the button $_counter times.'),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: Text('+'),
      ),
    );
  }
}
```

Fonte: Sato, 2022

Estrutura de um scaffold (estilo Venn)



Fonte: Sato, 2022

Widgets de Texto e Imagem

Widget de texto

- Um dos blocos mais básicos
- Vários *widgets* mais avançados já possuem propriedades de texto

Widget de texto

- **Text** (String texto)

19

Propriedades de um *widget* de texto

- **Style**: Estilo do texto, inclui fonte, tamanho da fonte, negrito, itálico, cor, etc.
- **Locale**: Seleciona uma fonte diferente dependendo da localização passada
- **MaxLines**: Quantidade máxima de linhas

20

Propriedades de um *widget* de texto

- **Overflow**: Ajuste para textos longos
- **textAlign**: Escolhe um alinhamento horizontal para o texto
- **textDirection**: Altera a direcionalidade de textos para textos que devem ser invertidos

21

Widget de texto

```
Text(  
  'Exemplo de Texto',  
  textAlign: TextAlign.center,  
  style: TextStyle(  
    fontWeight: FontWeight.bold,  
    fontSize 18.0  
  ))
```

22

Widget de imagem

- Outro *widget* bastante básico
- Muitas formas de se carregar uma imagem, a depender do local de onde a imagem será carregada

23

Widget de imagem

- **Image.asset**: ()
- Carrega uma imagem do aplicativo
- Exemplo: "assets/images/exemplo.jpg"

24

Widget de imagem

- **Image.network: ()**
- Carrega uma imagem da internet
Exemplo:
"https://upload.wikimedia.org/wikipedia/commons/3/31/Pizza_mezzo_a_mezzo.jpg"

25

Widget de imagem

- **Image.file: ()**
- Carrega uma imagem do aparelho do usuário
- Exemplo: "/caminho/da/imagem.png"

26

Propriedades de um *widget* de imagem

- **Alignment:** Alinhamento da posição da imagem
- **ColorBlendMode:** Em conjunto com *color*, aplica um filtro na imagem
- **Color:** Seleciona uma cor para aplicação de filtro na imagem

27

Propriedades de um *widget* de imagem

- **fit:** Ajuste da imagem para encaixar em um espaço
- **height:** Altura da imagem
- **width:** Largura da imagem

28

Propriedades de um *widget* de imagem

- **loadingBuilder:** Barra de carregamento para a imagem
- **opacity:** Opacidade/Transparência da imagem

29

Propriedades de um *widget* de imagem

- **repeat:** Repete a imagem no caso de espaço sobrando (ideal para imagens que se repetem como azulejos)
- **semanticLabel:** Descrição para usuários de TalkBack e VoiceOver

30

Widgets de Botões e Organização

31

Widgets de botões

- Não há um único botão genérico, mas sim vários tipos de botões, que seguem as convenções de *Material Design*, da Google e iOS Cupertino, da Apple
- Alguns exemplos mais usados:

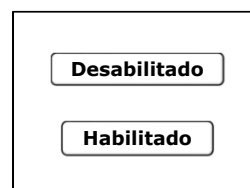
32

Widgets de botões

- ***ElevatedButton***: Botão retangular simples com animação ao ser pressionado

33

Widgets de botões



Fonte: Sato, 2022

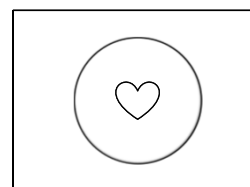
34

Widgets de botões

- ***floatingActionButton***: Botão arredondado que flutua sobre a interface

35

Widgets de botões



Fonte: Sato, 2022

36

Widgets de botões

- **`textButton`**: Botão em forma de texto

37

Widgets de botões

Desabilitado

Habilitado

Fonte: Sato, 2022

38

Widgets de botões

- **`outlinedButton`**: Botão de texto com um contorno

39

Widgets de botões

Texto com Contorno

Fonte: Sato, 2022

40

Widgets de botões

- **`iconButton`**: Botão em forma de imagem do tipo ícone

41

Widgets de botões



Volume: 40

Fonte: Sato, 2022

42

Propriedades de *widgets* de botões

- Nem todas as propriedades são aplicáveis a todos os tipos de botões
- **Color:** Define a cor do botão
- **Icon:** Define o ícone do botão
- **Label:** Define o texto do botão

43

Widgets de organização

- Os *widgets* quase sempre devem estar organizados de alguma forma
- Os dois meios mais comuns de se organizar *widgets* são na horizontal e na vertical, na forma de linhas e colunas

44

Widgets de organização: *Row*

- *Row* é uma linha
- Organiza os seus *widgets* em linha em ordem de declaração
- Exemplo:

45

Widgets de organização: *Row*

```
Row(  
  children: <Widget>[  
    Text('Widget 1'),  
    Icon(Icons.thumb_up),  
    Text('Widget 3')  
  ]  
)
```

Widget 1  Widget 3

Fonte: Sato, 2022

46

Widgets de organização: *Column*

- *Column* é uma Coluna
- Organiza os seus *widgets* em coluna, em ordem de declaração
- Exemplo:

47

Widgets de organização: *Column*

```
Column(  
  children: <Widget>[  
    Text('Widget 1'),  
    Icon(Icons.thumb_up),  
    Text('Widget 3')  
  ]  
)
```

Widget 1]

Widget 3

Fonte: Sato, 2022

48

Gestos do Usuário

49

Widget de detecção de gestos

- A interação com o usuário é uma peça chave para um bom aplicativo
- Uma das coisas mais comuns que um usuário faz quando não consegue interagir com o aplicativo é tentar fazer gestos na tela

50

Widget de detecção de gestos

- Um detector de gestos é um *widget* no seguinte formato:

```
GestureDetector(  
  função: () {  
    // código que será executado  
  }  
)
```

51

Widget de detecção de gestos

- A função do detector pode ser uma das diversas condições de gesto possíveis. Alguns dos mais simples são:
- **OnTap**: Detecta toque simples
- **OnDoubleTap**: Detecta toque duplo
- **OnLongPress**: Detecta toque longo

52

Widget de detecção de gestos

- É possível detectar também movimentos na horizontal e vertical

53

Widget de detecção de gestos

- **OnHorizontalDragDown**: Detecta quando um gesto pode iniciar um arrasto horizontal
- **OnHorizontalDragStart**: Detecta quando um arrasto horizontal inicia
- **OnHorizontalDragUpdate**: Detecta quando se está arrastando horizontalmente
- **OnHorizontalDragEnd**: Detecta quando um arrasto na horizontal termina

54

Widget de detecção de gestos

- ***OnVerticalDragDown***: Detecta quando um gesto pode iniciar um arrasto vertical
- ***OnVerticalDragStart***: Detecta quando um arrasto vertical inicia
- ***OnVerticalDragUpdate***: Detecta quando se está arrastando verticalmente
- ***OnVerticalDragEnd***: Detecta quando um arrasto na vertical termina

55

Widget de detecção de gestos

- Se o aparelho permitir, também é possível detectar pressão, colocada no toque da tela

56

Widget de detecção de gestos

- ***OnForcePressStart***: Início da pressão suficiente
- ***OnForcePressUpdate***: Mantém pressão suficiente
- ***OnForcePressPeak***: Atinge o máximo da detecção de pressão
- ***OnForcePressEnd***: Detecta quando o toque não estiver mais em contato com a tela

57

Widget de detecção de gestos

- Aplicativo de exemplo

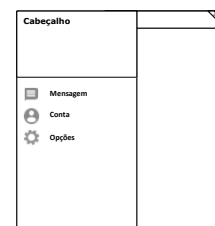
58

Widget de Drawer

- ***Drawer*** é uma “gaveta”
- Um ***drawer*** é um aplicativo que já vem com um detector de gestos imbutido

59

Widget de Drawer



Fonte: Sato, 2022

60

Widget de Drawer

```
Drawer: Drawer(  
  //widgets que fazem parte do drawer  
)
```

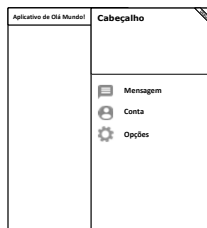
61

Widget de endDrawer

- **endDrawer** é uma “gaveta” que vem pelo final da tela

62

Widget de Drawer



Fonte: Salto, 2022

63

Propriedades de um widget de drawer

- **OpenDrawer**: Abre o *drawer* de um *scaffold*
- **OpenEndDrawer**: Abre o *endDrawer* de um *scaffold*
- **Navigator.pop**: Fecha um *drawer* que estiver aberto

64

Propriedades de um widget de drawer

- **DrawerEnableOpenDragGesture**: Se definido como “falso”, desabilita abrir um *drawer* com gestos
- **EndDrawerEnableOpenDragGesture**: Se falso, desabilita abrir um *endDrawer* com gestos

65

Widgets de gestos

- São bastante importantes para manter a imersão do usuário
- Utilize sempre que possível, como uma opção de navegação

66